



عملي

5



11



1650 sp

كلية الهندسة المعلوماتية

السنة الثانية

استدعاءات النظام في لغة التجميع MIPS

م. عبير الميا



RB Informatcs; 08/06/2026

بنیان الحاسوب ا

استدعاءات النظام

ماذا نعني باستدعاءات النظام؟

هي الطريقة التي تجعل البرنامج يتكلم مع نظام التشغيل (يعني تقدر تطبع، تقرأ إدخال من المستخدم، تخصص ذاكرة، وهكذا)

خطوات تنفيذ أي System Call:

1. نقوم بتحميل رقم الخدمة في السجل \$v0
2. حيث ان لكل وظيفة رقم معين.
3. نحمل المعاملات arguments (إن وجدت) في السجلات من \$a0 حتى \$a3
4. ثم ننفذ التعليمة syscall والتي ستقوم بتنفيذ الاستدعاء المطلوب.

جدول اهم الاستدعاءات (للحفظ):

الخدمة	رقم الاستدعاء
Print int	1 •
Print string	4 •
Read integer	5 •
Exit	10 •

شرح كل استدعاء :

1. Print int

عمله : طباعة رقم صحيح على الشاشة

كيفية استخدامه:

عم قول للنظام اسمع يا نظام بدي اطبع رقم $Li \$v0, 1$

حط الرقم الي بدي اطبعو والي هو 70 بالسجل \$a0 $Li \$a0, 70$

نفذ المطلوب $Syscall$

ملاحظة: العدد المراد طباعته يجب ان يكون في السجلات \$a0 دائما

Print string .2

عمله: لطباعة نص على الشاشة

كيفية استخدامه:

.data

string "Hello".asciiz Msg: # عرف النص بقسم ال data واستخدمنا .asciiz لتعريف الرسالة من نوع

.text

li \$v0,4 # تعليمة مشان قول للنظام بدي اطبع نص

La \$a0,Msg # حط عنوان النص ب \$a0

syscall # نفذ

ملاحظة: اذا هنا نضع عنوان النص من الذاكرة وليس النص كما فعلنا بالأعلى مع الأرقام أي نضع فقط العنوان (نحمل العنوان) لذلك ستخدم la وليس li

read integer .3

عمله: يوقف البرنامج و ينتظر المستخدم يدخل عدد ثم يحفظه ب \$v0

كيفية استخدامه:

li \$v0,5 #نخبر النظام انو بدنا نقرأ عدد

syscall #انتظر المستخدم وبعد ما يدخل العدد النتيجة تلقائيا بتروح ل \$v0

move \$t0,\$v0 #انقل العدد المدخل ل \$t0 مشان لو النظام رجع استخدم \$v0 لشي ثاني

exit .4

عمله: ينهي البرنامج. بدونيه ممكن أن يكمل البرنامج عمله لكن يعطي نتائج خاطئة

كيفية استخدامه:

li \$v0,10 # اخبر النظام ان البرنامج انتهى

syscall

هاتان التعليمتان يوضعو اخر البرنامج مهما كان عمله.

```

.data
    n: .asciiz "Enter N:" # النص اللي رح ينطبع

.text
main:
    li $v0, 4           # خدمة طباعة نص
    la $a0, n           # تحميل عنوان النص
    syscall             # اطبع "Enter N:"

    li $v0, 5           # خدمة قراءة عدد
    syscall             # $v0 القيمة المُدخلة راحت لـ

    move $t0, $v0       # $t0 انقل العدد لـ
    sub $t0, $zero, $t0 # $t0 = 0 - $t0 (يعني سالب العدد)

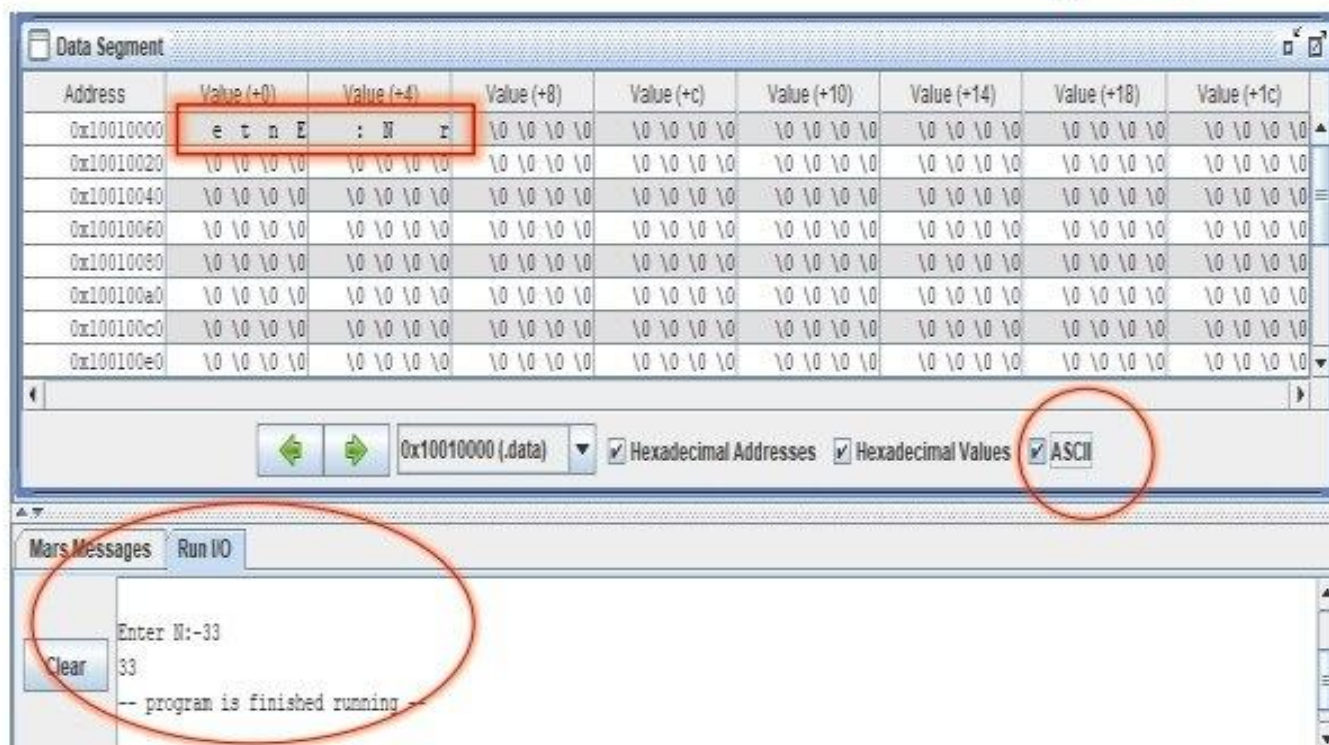
    move $a0, $t0       # $a0 حطّ الناتج في
    li $v0, 1           # خدمة طباعة عدد
    syscall             # اطبع الناتج

    li $v0, 10          # خدمة الإنهاء
    syscall

```



التنفيذ على برنامج المحاكاة:



تعليمات النمط (J type) :

لدينا تعليمتين فقط من هذا النوع ز و jal

تركيبية التعليمة في الذاكرة:

6 bits (OP)

26 bits (Address)

[L1 ز:](#)

قفز مباشرة للعنوان

[L1 jal ز:](#)

تكافئ التعليمتين:

jal ز

PC+4 ← \$ra (حفظ عنوان العودة في السجل \$ra)

طيب شو يعني PC+4

لانو لما يتم تنفيذ jal func, المعالج بحفظ عنوان التعليمة اللي بعدها في \$ra, لانه لما ينتهي التابع لازم يرجع

لها المكان بالضبط

• jr (jump register)

للقفز الى العنوان المخزن في سجل معين (غالبا تستخدم مع \$ra للعودة من التابع

jr \$ra

استدعاء التوابع Procedures/Functions

المبدأ الاساسي في استدعاء التوابع

Caller (المستدعي)	Callee (المستدعى)
ضع المعاملات الي بدك ياها بالسجلات \$a0 ل \$a3 بعدين بنستدعي التليمة jal func يعني اقفز لعند التابع func و احفظ وين كنا في البرنامج الرئيسي داخل \$ra مشان نرجع هون بعدين (يعني تخيلو انو مثل لما نخط اشارة بالكتاب على الصفحة الي وقفتي عليها)	Func: ... كود التابع او التعليمات.... Jr \$ra ال jr مشان نرجع للتابع

تمرين: ما عمل البرنامج func وكيف يمكن استدعاؤه من البرنامج الرئيسي وما محتوى x بنهاية التنفيذ

<pre> .data n: .word 5 x: .space 4 .text main: la \$s0,n la \$s1,x lw \$a0,0(\$s0) jal func sw \$v0,0(\$s1) li \$v0,10 syscall </pre>	<pre> func : mult \$a0,\$a0 mflo \$v0 jr \$ra #return to main </pre>
---	--

سيقوم التابع func بحساب القوة التربيعية حيث انه سيضرب العدد في نفسه ويعيد الناتج.

n=5

x=5^2=25

ولصحة انهاء التابع قمنا باضافة استدعاء النظام الخاص بانهاء البرنامج الرئيسي :

Address	Code	Basic	
0x00400000	0x3c011001	lui \$1,4097	5: main: la \$s0, n
0x00400004	0x34300000	ori \$16,\$1,0	
0x00400008	0x3c011001	lui \$1,4097	6: la \$s1, x
0x0040000c	0x34310004	ori \$17,\$1,4	
0x00400010	0x8e040000	lw \$4,0(\$16)	7: lw \$a0, 0(\$s0)
0x00400014	0x0c100009	jal 0x00400024	8: jal func # call func
0x00400018	0xae220000	sw \$2,0(\$17)	9: sw \$v0,0(\$s1)
0x0040001c	0x2402000a	addiu \$2,\$0,10	11: li \$v0, 10
0x00400020	0x0000000c	syscall	12: syscall
0x00400024	0x00840018	mult \$4,\$4	15: mult \$a0, \$a0
0x00400028	0x00001012	mflo \$2	16: mflo \$v0
0x0040002c	0x03e00008	jr \$31	17: jr \$ra # return to main

التنفيذ على MARS:

return address register

\$ra 31 0x00400018

التمثيل الثنائي لتعليمة jal هو:

6 bits 26 bits

Op Address

Op=3=000011₂

حقل العنوان من ٢٦ بت ثنائياً:

00 0001 0000 0000 0000 0000 0000 1001

يضاف 00 للأدنى، و٤ بت من PC للأعلى فيتشكل العنوان:

0x00400024 وهو عنوان القفز إلى Func وقيمته: 0000 0000 0100 0000 0000 0000 0010 0100

تمرين: اثناء فحص تطبيق محمي بمفتاح تفعيل تمكنا من استخراج الجزء المسؤول عن فحص مفتاح الترخيص المطلوب تحديد الخوارزمية المستخدمة من اجل هذه الحماية

<pre>.data msg_get_id: .asciiz "Enter Your ID: " msg_get_key: .asciiz "Enter Key: " msg_success: .asciiz "\n Access Granted! System Unlocked.\n" msg_failed: .asciiz "\n Access Denied! Try Again.\n" .text li \$v0, 4 la \$a0, msg_get_id syscall # طباعة رسالة إدخال المعرف li \$v0, 5 # قراءة المعرف</pre>	<pre>syscall move \$t0, \$v0 li \$v0, 4 la \$a0, msg_get_key # طلب إدخال مفتاح التفعيل syscall li \$v0, 5 syscall # قراءة المفتاح المدخل \$t1, \$v0</pre>	<pre>sll \$t2, \$t0, 1 xori \$t2, \$t2, 15 beq \$t1, \$t2, success li \$v0, 4 la \$a0, msg_failed syscall j exit success: li \$v0, 4 la \$a0, msg_success syscall exit: li \$v0, 10 syscall</pre>
--	---	---

التعليق:

```

Mars Messages Run I/O
Enter Your ID: 10
Enter Key: 20

Access Denied! Try Again.

-- program is finished running --

Enter Your ID: 12
Enter Key: 23

Access Granted! System Unlocked.

-- program is finished running --

```

تمرين: للتدريب: ما عمل البرنامج func وكيف يمكن استدعاؤه من البرنامج الرئيسي وما محتوى x
بنهاية التنفيذ؟

<pre> .data n: .word 6 x: .space 4 .text main: la \$s0, n la \$s1, x lw \$a0, 0(\$s0) jal func # استدعي التابع sw \$v0, 0(\$s1) li \$v0, 10 syscall </pre>	<pre> func: andi \$t0, \$a0, 1 beq \$t0, \$zero, L1 li \$v0, 0 j done L1: li \$v0, 1 done: jr \$ra </pre>
--	---

سيقوم البرنامج func باختبار زوجية العدد في \$a0 اذا كان زوجيا يرجع \$v0=1 والا يرجع 0

n=6 x=1
n=7 x=0

تمرين للتدريب: ما هو عمل البرنامج func وكيف يمكن استدعاؤه من البرنامج الرئيسي func(5) وما محتوى \$s0 في نهاية التنفيذ؟

func: li \$t0,1 li \$v0,1 L1: blt \$a0,\$t0,L2 mul \$v0,\$v0,\$t0 addi \$t0,\$t0,1 j L1 L2: jr \$ra	main: li \$a0,5 jal func move \$s0,\$v0 li \$v0,10 syscall
---	--

عمل البرنامج func: ايجاد عاملي العدد fact: \$s0 = 5! = 120 = 0x78

تمرين للتدريب: اكتب بلغة التجميع MIPS تابع SWAP المقابل للبرنامج التالي بلغة C افترض ان \$a0 العنوان القاعدي للمصفوفة v[], \$a1=k, ما هي التعليمة الممكن استخدامها لاستدعاء هذا التابع من برنامج اخر؟

#in C	#in MIPS
<pre>void swap(int v[],int k) { int temp; temp=v[k]; v[k]=v[k+1]; v[k+1]=temp; }</pre>	<pre>swap: mul \$t0,\$a1,4 # \$t0=k*4 (sll \$t0,\$a1,2) add \$t0,\$t0,\$a0 # \$t0=v+k*4 lw \$t1,0(\$t0) # t1=v[k] lw \$t2,4(\$t0) # t2=v[k+1] sw \$t2,0(\$t0) # v[k]=\$t2 sw \$t1,4(\$t0) # v[k+1]=\$t1 jr \$ra # return</pre>



لاستدعاء هذا التابع من برنامج اخر نكتب: jal swap مثال على استدعاؤه:

```
.data
arr: .word 1,2,3,4,5
.text
la $a0,arr
li $a1,2
jal swap
li $v0,10
syscall
```

شرح الحل: اذا عنا تابع بلغة ال c وهو لتبديل بين عنصرين ولاننا نكتب تابع بلغة MIPS بيحاكي العمل نفسو طيب هون عنا مشكلة كبدائية انو كيف بدنا ل $v[k]$ في الذاكرة؟
بنعرف انو كل عنصر بالمصفوفة بياخذ 4 bytes فعنوان $v[k]$ هو

$$\text{عنوان } v[k] = \text{العنوان القاعدي} + (k \times 4)$$

يعني لو $k=2$ والمصفوفة ببدا عنوانها من 1000:

$v[0]$ عنوان 1000 , $v[1]$ عنوان 1004 , $v[2]$ عنوان 1008

خطوة 4 : lw \$t2, 4(\$t0)

نحمل $v[k+1]$ في — \$t2 لاحظ $\text{offset}=4$ لأن العنصر بعده ب 4 bytes

مثال $: \$t2 = v[3] = 4$

خطوة 5 : sw \$t2, 0(\$t0)

نكتب \$t2 يعني $v[k+1]$ في مكان $v[k]$

مثال $: v[2] = 4$

خطوة 6 : sw \$t1, 4(\$t0)

نكتب \$t1 يعني القيمة الأصلية لـ $v[k]$ في مكان $v[k+1]$

مثال $: v[3] = 3$

خطوة 7 : jr \$ra

ارجع للبرنامج الرئيسي

خطوة 1 : sll \$t0, \$a1, 2

إزاحة يسار بمقدار 2 = ضرب ب 4

يعني $\leftarrow \$t0 = k \times 4$ هاد هو الـ offset

مثال $: k=2 \rightarrow \$t0 = 2 \times 4 = 8$

خطوة 2 : add \$t0, \$t0, \$a0

نضيف الـ offset على العنوان القاعدي

يعني $\$t0 = \text{عنوان } v[k]$

مثال: عنوان المصفوفة=1000, $\$t0=8 \rightarrow \$t0=1008$

خطوة 3 : lw \$t1, 0(\$t0)

نحمل $v[k]$ في \$t1

مثال $: \$t1 = v[2] = 3$

محتوى ذاكرة المعطيات قبل التنفيذ						
Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)
0x10010000	0x00000001	0x00000002	0x00000003	0x00000004	0x00000005	0x00000000
0x10010020	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000

محتوى ذاكرة المعطيات بعد التنفيذ						
Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)
0x10010000	0x00000001	0x00000002	0x00000004	0x00000003	0x00000005	0x00000000
0x10010020	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000

تدريب: اكتب تابع يقوم بايجاد مربع العدد n الذي يدخله المستخدم وتابعه يقوم بايجاد مكعب العدد الذي يدخله المستخدم ايضا:

وفي البرنامج الرئيسي يطلب من المستخدم ادخال احد الخيارين (1و2) ليتم استدعاء احد التابعين من البرنامج الرئيسي بحسب الخيار المحدد

الحل:

<pre>.data msg_choice: .asciiz "Enter 1 for Square, 2 for Cube: " msg_num: .asciiz "Enter the number N: " msg_res: .asciiz "The result is: " msg_error: .asciiz "Invalid choice! Please run again.\n" .text main: li \$v0, 4 la \$a0, msg_choice syscall li \$v0, 5 syscall move \$t0, \$v0 beq \$t0, 1, do_square beq \$t0, 2, do_cube li \$v0, 4 la \$a0, msg_error syscall</pre>	<pre>J exit do_square: li \$v0, 4 la \$a0, msg_num syscall li \$v0, 5 syscall move \$a0, \$v0 jal Square move \$t1, \$v0 j print_result do_cube: li \$v0, 4 la \$a0, msg_num syscall li \$v0, 5 syscall move \$a0, \$v0</pre>	<pre>jal Cube move \$t1, \$v0 j print_result print_result: li \$v0, 4 la \$a0, msg_res syscall li \$v0, 1 move \$a0, \$t1 syscall exit: li \$v0, 10 syscall Square: mul \$v0, \$a0, \$a0 jr \$ra Cube: mul \$v0, \$a0, \$a0 mul \$v0, \$v0, \$a0 jr \$ra</pre>
--	--	--

ملاحظة اخيرة (غير مطلوبة بالعملي مطلوبة بالنظري)

في حال الاستدعاء المتداخل للتوابع (مثلا التابع func1 يستدعي func2) وذلك باستخدام تعليمة jal حيث يتم حفظ عنوان العودة (PC+4) عند كل استدعاء في \$ra وفي نهاية كل تابع يوجد jr \$ra للعودة الى البرنامج المستدعي. فكيف يتم تحديد أي من عناوين العودة هو المقصود عند تنفيذ jr \$ra ؟

الجواب: يتم استخدام ال stack لحفظ عنوان العودة قبل استدعاء كل تابع حتى لا يتم فقدان قيمته. نمو المكس نحو الاسفل (أي باتجاه العناوين الالدى). ويوجد السجل (\$sp Stack Pointer) والذي يؤشر الى عنوان قمة الستاك عند اضافة عنصر الى stack ينقص \$sp بمقدار 4- وعند ازالة عنصر يزداد بمقدار 4

فاذا اردنا استدعاء func2 داخل func1 تقوم بالتالي قبل الاستدعاء:

```
addi $sp,$sp,-4
```

```
sw $ra,0($sp)
```

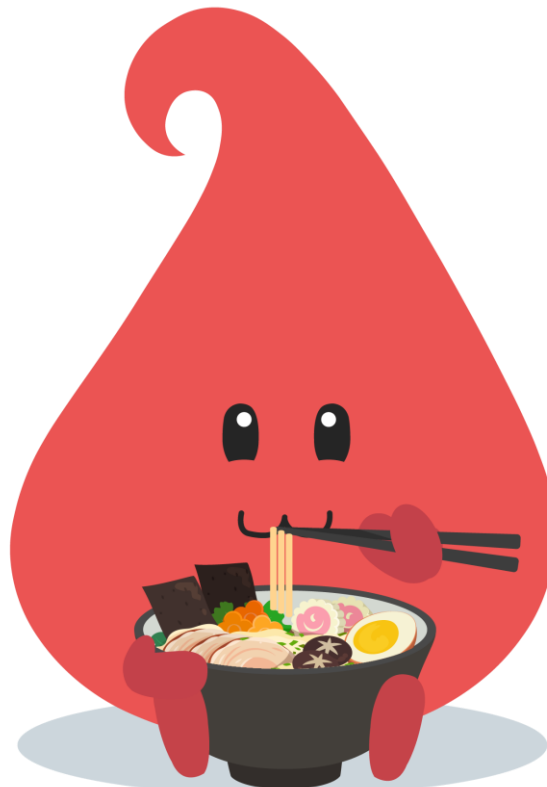
وبعد ذلك في func1, قبل ان نكتب jr \$ra نقوم ب:

```
lw $ra,0($sp)
```

```
addi $sp,$sp,4
```

```
jr $ra
```

keep going...



تمرين:

ليكن لدينا مصفوفات درجات طالب grades مخزنة في الذاكرة/ يطلب إدخالها، اكتب برنامج بلغة التجميع يقوم باستدعاء تابع studentGrades مهمته: المرور على درجات الطالب واختبار إذا كانت الدرجة أصغر تماما من 60 يزيد عدد مواد الرسوب عند الطالب بمقدار 1، بعد الانتهاء من معالجة جميع الدرجات يختبر إذا كان عدد مواد الرسوب أكبر أو يساوي 4 مواد يعتبر الطالب راسب ويطبع The student failed وإلا يطبع The student passed
(انتبه لاستخدام استدعاءات النظام syscall للإدخال والإخراج)

الحل:

```

1  .data
2  grades: .space 24
3  fail: .asciiz "The student failed"
4  pass: .asciiz "The student passed"
5
6  .text
7  la $t0, grades
8  li $s0, 0
9  # loop to get grades from user, loops 6 times
10 input:
11     beq $s0, 6, out
12     li $v0, 5
13     syscall
14     move $t5, $v0
15     sw $t5, 0($t0)
16     addi $t0, $t0, 4
17     addi $s0, $s0, 1
18     j input
19 out:
20     la $t0, grades
21     jal studentGrades # calling the function
22

```



تمرین:

اكتب برنامج بلغة التجميع يحاكي نظام مصرفي، بحيث يدخل المستخدم رصيد الحساب ورصيد السحب الذي يريد ان يسحبه منه ولديك دالة check تتحقق ما إذا كان السحب من مضاعفات العدد 10 والسحب اقل من الرصيد، اذا تمت العملية يطبع نجاح ويعيد رصيد الحساب بعد السحب والا يرجع القيمة (-1)

الحل:

```

1  .data
2      p_bal: .asciiz "Enter current balance: "
3      p_amt: .asciiz "Enter withdrawal amount: "
4      m_suc: .asciiz "Success! New Balance: "
5      m_err: .asciiz "Error: "
6  .text
7  main:
8      li $v0, 4 # 4 to print a string
9      la $a0, p_bal # loading p_bal address to $a0, so syscall can use it
10     syscall
11     li $v0, 5 # 5 to get integer from user
12     syscall
13     move $s0, $v0 # the entered integer gets saved in $v0, that's why we move it to another register
14     li $v0, 4
15     la $a0, p_amt
16     syscall
17     li $v0, 5
18     syscall
19     move $s1, $v0
20     move $a0, $s0
21     move $a1, $s1
22     jal Check # we used jal not j so we can use "jr $ra" later to get back here
23     #####

```



“Remember: The hardest choices require the strongest wills”